

WEEK-8 SKILL

1 . To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic

Aim : To implement variable expansion in a shell-like environment by detecting variable references, replacing them with their values, handling undefined variables, updating tokens, supporting nested variables, and testing the expansion logic.

Program :

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_OUTPUT 4096

void expand_variables(const char *input, char *output)
{
    int i = 0;
    int j = 0;

    while (input[i] != '\0' && j < MAX_OUTPUT - 1)
    {
        if (input[i] == '$')
        {
            i++;

            /* Handle ${VARIABLE} format */
            if (input[i] == '{')
            {
                i++;

                char variable[100];
                int k = 0;

                while (input[i] != '\0' &&
                    input[i] != '}' &&
                    k < 99)
                {
                    variable[k++] = input[i++];
                }

                variable[k] = '\0';

                if (input[i] == '}')
                    i++;

                char *value = getenv(variable);

                if (value != NULL)
                {
                    for (int x = 0;
                        value[x] != '\0' &&
                        j < MAX_OUTPUT - 1;
                        x++)
                    {

```

```
        output[j++] = value[x];
    }
    }
    else
    {
        printf("Undefined variable: %s\n", variable);
    }
}

/* Handle $VARIABLE format */
else if (isalpha((unsigned char)input[i]) ||
        input[i] == '_')
{
    char variable[100];
    int k = 0;

    while ((isalnum((unsigned char)input[i]) ||
            input[i] == '_') &&
            k < 99)
    {
        variable[k++] = input[i++];
    }

    variable[k] = '\0';

    char *value = getenv(variable);

    if (value != NULL)
    {
        for (int x = 0;
             value[x] != '\0' &&
             j < MAX_OUTPUT - 1;
             x++)
        {
            output[j++] = value[x];
        }
    }
    else
    {
        printf("Undefined variable: %s\n", variable);
    }
}
else
{
    output[j++] = '$';
}
}
```

```
strcpy(input, "$NAME is studying $COURSE in $YEAR");
expand_variables(input, output);

printf("Input : %s\n", input);
printf("Output: %s\n\n", output);


printf("Test 3: Undefined variable\n");

strcpy(input, "Value of $UNKNOWN");
expand_variables(input, output);

printf("Input : %s\n", input);
printf("Output: %s\n\n", output);


printf("Test 4: Braced variable\n");

strcpy(input, "Student: ${NAME}");
expand_variables(input, output);

printf("Input : %s\n", input);
printf("Output: %s\n\n", output);


printf("Test 5: Nested variable\n");

setenv("MESSAGE", "Welcome $NAME", 1);

strcpy(input, "$MESSAGE");
expand_variables(input, output);

printf("Input : %s\n", input);
printf("First expansion: %s\n", output);

char nested_output[MAX_OUTPUT];

expand_variables(output, nested_output);

printf("Nested expansion: %s\n\n", nested_output);

printf("==== EXPANSION TEST COMPLETED =====\n");

return 0;
}
```

Output :

```
sindhuja@SINDHUJA: ~/va X + v
sindhuja@SINDHUJA:~$ mkdir variable_expansion
sindhuja@SINDHUJA:~$ cd variable_expansion
sindhuja@SINDHUJA:~/variable_expansion$ pwd
/home/sindhuja/variable_expansion
sindhuja@SINDHUJA:~/variable_expansion$ NAME="Sindhuja"
sindhuja@SINDHUJA:~/variable_expansion$ COURSE="OSSP"
sindhuja@SINDHUJA:~/variable_expansion$ YEAR="2026"
sindhuja@SINDHUJA:~/variable_expansion$ echo $NAME
Sindhuja
sindhuja@SINDHUJA:~/variable_expansion$ echo $COURSE
OSSP
sindhuja@SINDHUJA:~/variable_expansion$ echo $YEAR
2026
sindhuja@SINDHUJA:~/variable_expansion$ echo "Student: $NAME"
Student: Sindhuja
sindhuja@SINDHUJA:~/variable_expansion$ echo "$NAME is studying $COURSE in $YEAR"
Sindhuja is studying OSSP in 2026
sindhuja@SINDHUJA:~/variable_expansion$ echo $UNKNOWN
Value = [$UNKNOWN]
sindhuja@SINDHUJA:~/variable_expansion$ if [ -z "$UNKNOWN" ]; then
> echo "UNKNOWN is undefined"
> fi
UNKNOWN is undefined
sindhuja@SINDHUJA:~/variable_expansion$ NAME="Sindhuja"
sindhuja@SINDHUJA:~/variable_expansion$ MESSAGE="Hello $NAME"
sindhuja@SINDHUJA:~/variable_expansion$ echo "$MESSAGE"
Hello Sindhuja
```

```
sindhuja@SINDHUJA:~/variable_expansion$ nano variable_expand.c
sindhuja@SINDHUJA:~/variable_expansion$ gcc variable_expand.c -o variable_expand
sindhuja@SINDHUJA:~/variable_expansion$ ls
variable_expand  variable_expand.c
sindhuja@SINDHUJA:~/variable_expansion$ ./variable_expand
===== VARIABLE EXPANSION PRACTICAL =====

Test 1: Basic variable expansion
Input : Hello $NAME
Output: Hello Sindhuja

Test 2: Multiple variables
Input : $NAME is studying $COURSE in $YEAR
Output: Sindhuja is studying OSSP in 2026

Test 3: Undefined variable
Undefined variable: UNKNOWN
Input : Value of $UNKNOWN
Output: Value of

Test 4: Braced variable
Input : Student: ${NAME}
Output: Student: Sindhuja

Test 5: Nested variable
Input : $MESSAGE
First expansion: Welcome $NAME
Nested expansion: Welcome Sindhuja

===== EXPANSION TEST COMPLETED =====
sindhuja@SINDHUJA:~/variable_expansion$
```

2 . To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

Aim : To identify shell built-in commands, create a dispatch table for those commands, execute built-in commands within the shell process, handle invalid commands, maintain shell state, and test the command-dispatch logic.

Program :

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX_INPUT 100
#define MAX_ARGS 10

/* Function declarations */
void builtin_pwd(char **args);
void builtin_cd(char **args);
void builtin_echo(char **args);
void builtin_exit(char **args);

/* Structure for dispatch table */
struct builtin_command
{
    char *name;
    void (*function)(char **args);
};

/* Dispatch table */
struct builtin_command builtins[] =
{
    {"pwd", builtin_pwd},
    {"cd", builtin_cd},
    {"echo", builtin_echo},
    {"exit", builtin_exit}
};

int builtin_count = 4;

/* pwd command */
void builtin_pwd(char **args)
{
    char cwd[1024];

    if (getcwd(cwd, sizeof(cwd)) != NULL)
    {
        printf("%s\n", cwd);
    }
    else
    {
        f
```

GNU nano 7.2

```
{
    perror("pwd");
}

/* cd command */
void builtin_cd(char **args)
{
    if (args[1] == NULL)
    {
        printf("cd: missing directory\n");
        return;
    }

    if (chdir(args[1]) != 0)
    {
        perror("cd");
    }
}

/* echo command */
void builtin_echo(char **args)
{
    int i = 1;

    while (args[i] != NULL)
    {
        printf("%s", args[i]);

        if (args[i + 1] != NULL)
        {
            printf(" ");
        }

        i++;
    }

    printf("\n");
}

/* exit command */
void builtin_exit(char **args)
{
|
```

GNU nano 7.2

```
{
    printf("Exiting shell...\n");
    exit(0);
}

/* Find command in dispatch table */
int find_builtin(char *command)
{
    int i;

    for (i = 0; i < builtin_count; i++)
    {
        if (strcmp(command, builtins[i].name) == 0)
        {
            return i;
        }
    }

    return -1;
}

/* Main function */
int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];
    char *token;
    int argc;
    int index;

    printf("==== BUILT-IN COMMAND SHELL ==== \n");
    printf("Available commands: pwd, cd, echo, exit \n \n");

    while (1)
    {
        printf("myshell> ");

        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            break;
        }

        /* Remove newline */
    }
}
```

```
        break;
    }

    /* Remove newline */
    input[strcspn(input, "\n")] = '\0';

    /* Ignore empty input */
    if (strlen(input) == 0)
    {
        continue;
    }

    /* Tokenize input */
    argc = 0;
    token = strtok(input, " ");

    while (token != NULL && argc < MAX_ARGS - 1)
    {
        args[argc++] = token;
        token = strtok(NULL, " ");
    }

    args[argc] = NULL;

    /* Find command in dispatch table */
    index = find_builtin(args[0]);

    if (index != -1)
    {
        printf("[Built-in command detected]\n");

        /* Execute command in current process */
        builtins[index].function(args);
    }
    else
    {
        printf("Invalid command: %s\n", args[0]);
    }
}

return 0;
}
```

^G Help	^O Write Out	^W Where Is	^K Cut
^X Exit	^R Read File	^_ Replace	^U Paste

Output :

```
sindhuja@SINDHUJA: ~/bu x + v
sindhuja@SINDHUJA:~$ mkdir builtin_practical
sindhuja@SINDHUJA:~$ cd builtin_practical
sindhuja@SINDHUJA:~/builtin_practical$ pwd
/home/sindhuja/builtin_practical
sindhuja@SINDHUJA:~/builtin_practical$ nano builtin_shell.c
sindhuja@SINDHUJA:~/builtin_practical$ gcc builtin_shell.c -o builtin_shell
sindhuja@SINDHUJA:~/builtin_practical$ ./builtin_shell
===== BUILT-IN COMMAND SHELL =====
Available commands: pwd, cd, echo, exit

myshell> pwd
[Built-in command detected]
/home/sindhuja/builtin_practical
myshell> echo Hello World
[Built-in command detected]
Hello World
myshell> echo OSSP Practical
[Built-in command detected]
OSSP Practical
myshell> pwd
[Built-in command detected]
/home/sindhuja/builtin_practical
myshell> cd /tmp
[Built-in command detected]
myshell> pwd
[Built-in command detected]
/tmp
myshell> hello
Invalid command: hello
myshell> xyz
Invalid command: xyz
myshell> cd
[Built-in command detected]
cd: missing directory
myshell> exit
[Built-in command detected]
Exiting shell...
sindhuja@SINDHUJA:~/builtin_practical$ |
```